МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки»

Лабораторная работа №1
по курсу «Программирование графических процессоров»

Освоение программного обеспечения для работы с технологией CUDA. Примитивные операции над векторами.

Выполнил: Федорова Екатерина Васильевна
Группа: М8О-408Б-23
Преподаватель: А.Ю. Морозов

Москва, 2026

Условие

Цель работы: ознакомление и установка программного обеспечения для работы с программно-аппаратной архитектурой параллельных вычислений (CUDA). Реализация примитивной операции над векторами: поэлементное нахождение максимума векторов.

Вариант: 5. Входные данные: на первой строке задано число n - размер векторов. В следующих 2-х строках, записано по n вещественных чисел - элементы векторов. Выходные данные: n чисел - результат поэлементного нахождения максимума исходных векторов.

Программное и аппаратное обеспечение

GPU: NVIDIA GeForce RTX 4050 Laptop GPU
Compute capability: 8.9
Общая память: 6141 MiB, занятая память: 15 MiB, свободная память: 5759 MiB
Графическая память: 5772 MB
Разделяемая память (по умолчанию): 48 KB
Разделяемая память (opt-in, на блок): 99 KB
Разделяемая память (на мультипроцессор): 100 KB
Константная память: 64 KB
Кол-во регистров на блок: 65536
Кол-во регистров на мультипроцессор: 65536
Макс. кол-во потоков на блок: 1024
Макс. размерность блока: 1024 x 1024 x 64
Макс. кол-во потоков на мультипроцессор: 1536
Макс. кол-во блоков на мультипроцессор: 24
Максимальный размер сетки: 2147483647 x 65535 x 65535
Мультипроцессоры: 20

CPU: AMD Ryzen 7 7735HS with Radeon Graphics
Архитектура: x86_64
Address sizes: 48 bits physical, 48 bits virtual
CPU(s): 16
Потоков на ядро: 2
Ядер на сокет: 8
Сокетов: 1
CPU max MHz: 4829,0000
CPU min MHz: 400,0000
Виртуализация: AMD-V
Caches (sum of all):
L1d: 256 KiB (8 instances)
L1i: 256 KiB (8 instances)
L2: 4 MiB (8 instances)
L3: 16 MiB (1 instance)

RAM

	Всего	Занято	Свободно	Общая	Буф/врем.	Доступно
Память	14Gi	7,0Gi	2,3Gi	166Mi	5,9Gi	7,8Gi
Подкачка	2,0Gi	1,2Gi	795Mi			

SSD: DGSM4512GM63T, 476,9G, 0 disk
Файл.система: /dev/nvme0n1p3
Тип: ext4
Размер: 465G
Использовано: 107G

ОС и ПО
Название: Ubuntu 24.04.4 LTS
Версия: 24.04.4 LTS (Noble Numbat)
Ядро: 6.8.0-142-generic
NVIDIA-SMI: 595.84
Driver Version: 595.84
CUDA Version: 13.2
CUDA Toolkit: 12.0 (nvcc V12.0.140)
VSCode: 1.138.0, x64

Метод решения

В данной работе реализована задача поэлементного нахождения максимума двух векторов с использованием технологии CUDA. На вход программы подаётся размер векторов n и два вектора вещественных чисел a и b. Необходимо сформировать результирующий вектор result, каждый элемент которого определяется по формуле:

$$result_i = max(a_i, b_i), i = 0 \ldots n-1.$$

Для решения задачи реализованы две версии алгоритма: последовательная на центральном процессоре (CPU) и параллельная на графическом процессоре (GPU) с использованием CUDA. Обе версии используют тип данных double для хранения элементов векторов.

Последовательный алгоритм (CPU) заключается в последовательном переборе элементов входных векторов. На каждой итерации сравниваются соответствующие элементы векторов, после чего больший из них записывается в результирующий вектор. Каждый элемент обрабатывается один раз, поэтому временная сложность алгоритма составляет O(n).

Параллельный алгоритм (GPU) реализован с использованием технологии CUDA. Вычисления выполняются CUDA-ядром, которое запускается на графическом процессоре. Обработка элементов распределяется между потоками GPU: каждый поток вычисляет максимум для одного или нескольких элементов входных векторов.

Для распределения вычислений используется сетка из 1024 блоков по 256 потоков в каждом. Таким образом, общее количество логических потоков, задействованных при запуске ядра, составляет 262.144 и не зависит от размера входных данных. Для обработки векторов произвольного допустимого размера используется цикл с шагом, равным общему количеству потоков (grid-stride loop). Если количество элементов превышает количество потоков, каждый поток обрабатывает несколько элементов с постоянным шагом.

Перед запуском ядра входные векторы копируются из оперативной памяти компьютера в глобальную память GPU. После завершения вычислений результирующий вектор копируется обратно в оперативную память для последующего вывода.

Для оценки производительности измеряется время непосредственно выполнения CUDA-ядра с помощью событий CUDA (cudaEvent). Время последовательного алгоритма на CPU измеряется средствами библиотеки std::chrono. В обоих случаях измерение проводится отдельно от ввода, вывода и передачи данных между CPU и GPU, что позволяет сравнить длительность самих вычислений.

Описание программы

Состав проекта

Файл	Назначение
lab1_cpu.cpp	эталонная CPU-реализация, использующая последовательный цикл и замер времени через `std::chrono`
lab1.cu	простейшая GPU-версия без тайминга для отправки в чекер
lab1_timer.cu	GPU-версия с таймингом через CUDA events, основная для отчёта

Также в проекте присутствуют четыре файла с входными данными: test3.txt, test100.txt, test1000000.txt и test33000000.txt. Они используются для замеров времени при n = 3, n = 100, n = 1.000.000 и n = 33.000.000 соответственно.

Основные типы данных

В качестве типа элементов массивов используется double. Для замера времени на устройстве используется тип cudaEvent_t — события CUDA. Код возврата всех runtime-функций CUDA имеет тип `cudaError_t` и обёрнут в функцию `checkCuda`.

Реализованные ядра

В программе реализовано единственное ядро findMax. Его сигнатура:

```cpp
__global__ void findMax(const double* __restrict__ a,
                        const double* __restrict__ b,
                        double* __restrict__ result,
                        int n)
```

Параметры ядра:

- a, b: входные массивы в глобальной памяти GPU;
- result: выходной массив;
- n: длина массивов.

Квалификаторы:

- __global__: ядро, вызывается с хоста, исполняется на устройстве;
- const __restrict__ для входных указателей: компилятор может агрессивнее оптимизировать (нет пересечений указателей между a, b и result).

Вспомогательные функции

Функция checkCuda:

```cpp
bool checkCuda(cudaError_t error, const char* operation);
```

Универсальная проверка кода возврата CUDA. При ошибке печатает имя операции и текст ошибки (cudaGetErrorString) в stderr и возвращает false. Позволяет в main строить цепочки проверок без потери читаемости.

Функции closeCPU и closeGPU:
```cpp
template <typename... Pointers>
void closeCPU(Pointers*... pointers);   // delete[] для каждого
template <typename... Pointers>
void closeGPU(Pointers*... pointers);   // cudaFree для каждого
```
Вариативные шаблоны, принимающие произвольное число указателей. Применяются для централизованной очистки ресурсов в любой ветке выхода.

Общая структура main в лабораторных

1. Настройка ввода-вывода: sync_with_stdio(false) и cin.tie(nullptr) для ускорения ввода; scientific для формата вывода.
2. Чтение: n с проверкой диапазона $0<n<2^{25}$ и массивов.
3. (для CPU) Выделение памяти на GPU и копирование массивов.
4. Создание событий времени start и stop.
5. Запуск ядра или последовательного алгоритма.
6. (для CPU) Копирование результата и вычисление затраченного времени.
7. Вывод результатов.
8. Освобождение ресурсов: событий, GPU-памяти, CPU-памяти.

Отладочный вывод и тестирование

В lab1_timer.cu массив результатов не печатается (соответствующий цикл закомментирован) — на больших n у меня вис компьютер и вылетал VSCode.
Для проверки используются четыре файла.

Результаты

n	Кол-во блоков	Кол-во потоков (в блоке)	Время GPU [ms]	Время CPU [ms]
3	1	32	1.7286e-01	1.2000e-04
	2	32	1.2384e-01	
	4	64	1.7306e-01	
	8	128	1.2608e-01	
	16	256	1.3190e-01	
	32	512	1.2493e-01	
	64	1024	1.2390e-01	
	128	256	1.2573e-01	
	256	256	1.2186e-01	
	512	512	1.2250e-01	
	1024	1024	1.3398e-01	
100	1	32	1.7398e-01	1.1920e-03
	2	32	1.7539e-01	
	4	64	1.8650e-01	
	8	128	2.2323e-01	
	16	256	1.7818e-01	
	32	512	1.7731e-01	
	64	1024	2.8365e-01	
	128	256	2.5622e-01	
	256	256	2.0787e-01	
	512	512	1.8550e-01	
	1024	1024	1.8022e-01	
1.000.000	1	32	1.9303e+00	1.4114e+01
	2	32	1.1723e+00	
	4	64	3.9328e-01	
	8	128	2.3750e-01	
	16	256	2.0454e-01	
	32	512	2.0576e-01	
	64	1024	1.8365e-01	
	128	256	1.8742e-01	
	256	256	1.9757e-01	
	512	512	1.9626e-01	

	1024	1024	2.1190e-01	
33.000.000	1	32	9.9055e+01	2.9680e+02
	2	32	5.2477e+01	
	4	64	1.6343e+01	
	8	128	6.0702e+00	
	16	256	4.6808e+00	
	32	512	4.6418e+00	
	64	1024	4.5822e+00	
	128	256	4.5491e+00	
	256	256	4.5550e+00	
	512	512	4.5602e+00	
	1024	1024	4.5694e+00	

Выводы

Область применения

Поэлементный максимум — тривиальная задача, лежащая в основе множества задач: ReLU и max-pooling в нейросетях, blend-режимы в графике, clipping сигналов в ЦОС, payoff опционов в финансах, поэлементное сравнение изображений. Отсутствие зависимостей по данным делает операцию идеальным кандидатом для SIMT-распараллеливания.

Сложность

O(n) по времени и памяти. При реализации на CUDA пришлось решить несколько типовых проблем:

- grid-stride loop вместо «один поток — один элемент», чтобы не привязываться к размеру сетки;
- обработка ошибок CUDA через обёртку checkCuda вокруг всех runtime-вызовов;
- замер времени через CUDA events, а не std::chrono;
- управление памятью через вариативные шаблоны.

Одной из особенностей задачи является значительная роль накладных расходов при небольшом размере входных данных. При малых n время выполнения вычислений на GPU мало по сравнению с затратами на запуск ядра и синхронизацию. Поэтому параллельная реализация не всегда эффективнее последовательной.

Сравнение результатов

При n = 3 и n = 100 время вычислений на CPU значительно меньше времени выполнения CUDA-ядра. Для n = 3 ускорение CPU относительно GPU составляет приблизительно 1000–1440 раз, а для n = 100 — около 146–238 раз в зависимости от конфигурации GPU. Это объясняется тем, что вычислительная нагрузка при таких размерах крайне мала, в то время как запуск ядра и синхронизация требуют дополнительных затрат. При этом различия между конфигурациями GPU на малых размерах не позволяют достоверно определить оптимальное количество блоков и потоков.

При n = 1.000.000 параллельная реализация показывает существенное преимущество по времени выполнения самого вычислительного ядра. Наилучший результат в проведённых измерениях составил для конфигурации из 64 блоков по 1024 потока. При времени CPU это соответствует ускорению примерно в 77 раз. При этом конфигурации с большим количеством блоков и потоков не демонстрируют постоянного улучшения производительности, что указывает на необходимость подбора параметров запуска с учётом размера задачи и архитектуры GPU.

При n = 33.000.000 время CPU составило около 296,8 мс, тогда как наилучшее измеренное время GPU составило приблизительно 4,555 мс для конфигурации из 256 блоков по 256 потоков. Ускорение составляет около 65 раз. Время выполнения для конфигураций с 16 блоками по 256 потоков и выше находится в сравнительно узком диапазоне. Это показывает, что после достижения достаточного уровня параллелизма

дальнейшее увеличение количества потоков не обязательно приводит к заметному повышению производительности.

В таблице наблюдаются колебания времени выполнения между конфигурациями. Они могут быть связаны с изменением частоты GPU, планированием выполнения блоков, состоянием кэшей, системными накладными расходами и погрешностью измерений. Поскольку каждый вариант запускался однократно, полученные значения следует рассматривать как ориентировочные. Для повышения достоверности результатов рекомендуется выполнять несколько повторных измерений каждой конфигурации после предварительного прогрева GPU и использовать медианное время.

Следует учитывать, что измерения GPU отражают только время выполнения ядра и не включают затраты на передачу входных данных на видеокарту и возврат результата. Поэтому полученное ускорение не является ускорением всей программы относительно CPU.

Итог

Проведённые измерения показывают, что эффективность параллельной реализации зависит от размера входных данных. Для очень малых векторов последовательный алгоритм на CPU предпочтительнее, поскольку накладные расходы GPU превышают выигрыш от распараллеливания. На крупных входных данных CUDA-ядро демонстрирует существенное преимущество по времени вычислений, достигая ускорения приблизительно в 65–77 раз в рассмотренных экспериментах.

Эксперименты также показывают, что увеличение количества блоков и потоков не всегда приводит к ускорению. Для определения оптимальной конфигурации необходимо учитывать размер задачи и характеристики графического процессора, а также проводить повторные измерения. Для оценки эффективности всей программы в дальнейшем следует дополнительно учитывать время передачи данных между CPU и GPU.